Proj

March 25, 2026

Probabilistic Forecasting of Cryptocurrency Returns

COMP 3009 — Mathematics for Data Science & AI

Nick Brady and Emily Suarez

1 Introduction: Introduction and Project Objective

In this project, we ask the practical research question: Does adaptive volatility learning substantially improve predictive uncertainty in noisy, high-frequency markets, or does the additional model complexity yield minimal gains?

To answer this question, we will use one-step-ahead probabilistic forecasting for 1-minute Bitcoin prices, based on BTC-USD candle data from Coinbase. Two models will be compared: a static Gaussian baseline with fixed volatility and a Bayesian predictive model that updates sequentially. Markets are inherently noisy, so the goal is not perfect prediction but better-calibrated uncertainty. Instead of predicting a single next price, the goal is to estimate the full predictive distribution of the next return and map that uncertainty back into a price forecast.

In real trading or risk settings, underestimating uncertainty can lead to overconfidence, which becomes especially costly during fast market moves. A model that looks acceptable on average can still fail when volatility spikes.

Those models show a key tradeoff: Bayesian Volatility improves uncertainty but can lag during rapid market changes, highlighting a fundamental concept of stability and responsiveness in sequential modeling

The main takeaway is that Bayesian updating provides slightly better uncertainty quantification, but the gains are limited and do not address the central failure. The underlying issue is the trade-off between stability and responsiveness. But both models remain vulnerable when market conditions change unexpectedly, which reinforces the central challenge is not just accuracy, but how a model balances stability with responsiveness.

2 Data and Structure: Data Description

2.0.1 Data Collection and Setup

This analysis uses the Coinbase Exchange API and 1-minute BTC/USD candlestick data for 1 year. Each candle contains open, high, low, close, and volume information, providing a sequential record of market behavior at a fine time resolution. In each candle, only the closing price is used to compute returns, establishing a consistent definition of price change.

Bitcoin was chosen for two reasons: it trades continuously, providing an environment for studying high-frequency uncertainty, and it offers 1-minute data that is noisy enough to stress-test whether adaptive volatility modeling can outperform a fixed-volatility baseline.

This section describes the global settings used throughout the project, including the data source, predefined time window, random seed, forecast settings, and helper functions for downloading and cleaning canonical candle data.

After cleaning and preprocessing, the dataset contains no missing values in the variables used for analysis.

2.0.2 Example of Raw Candle Data

The data is structured as a high-frequency time series, where each observation represents a 1-minute snapshot of market activity. Each row includes:

- open: price at the start of the interval
- high: highest price within the interval
- low: lowest price within the interval
- close: price at the end of the interval
- volume: traded volume during the interval

These features describe the price movement within each interval, forming a sequential record of market behavior over time.

2.0.3 Table 1 - Raw Candle Data

[3]:

	time	low	high	open	close	volume
0	1740355200	96266.23	96320.53	96266.23	96281.16	4.040833
1	1740355260	96281.15	96328.43	96281.15	96283.67	1.901450
2	1740355320	96214.67	96283.66	96283.66	96214.67	4.723413
3	1740355380	96196.74	96225.04	96215.32	96212.25	0.966086
4	1740355440	96212.25	96268.93	96212.25	96265.17	2.592311

3 From Prices to Returns: Transformation to Log Returns

The raw candlestick closing prices are transformed into log returns, allowing the models to focus on relative changes rather than raw price levels.

The log return at time t is defined as:

$$r_t = \log\left(\frac{P_t}{P_{t-1}}\right)$$

where P_t is the closing price at time t.

Price series are non-stationary and change scale over time. By transforming prices into log returns, the time series becomes more stable, making its behavior more consistent and better suited for probabilistic modeling.

This transformation also creates a direct connection between the statistical model and the forecasting step. Once a predictive distribution for returns is obtained, it can be mapped back into a predictive distribution for future prices.

3.0.1 Table 2 - Returns Data Sample

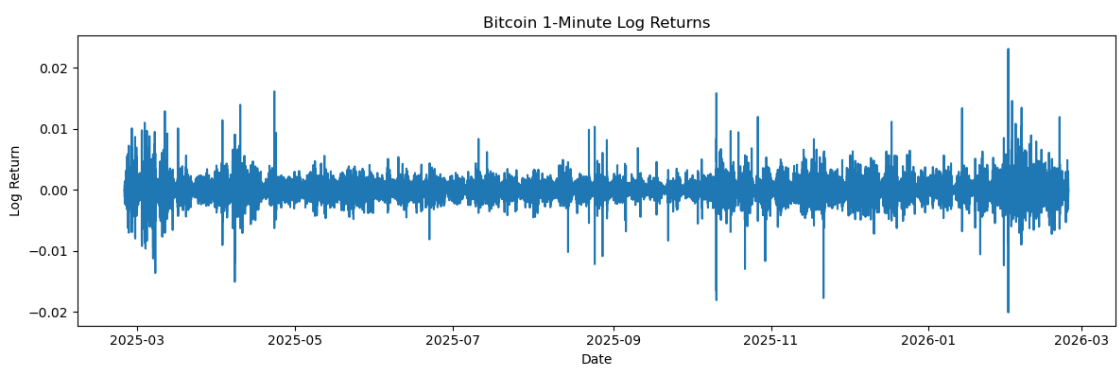
[4]:

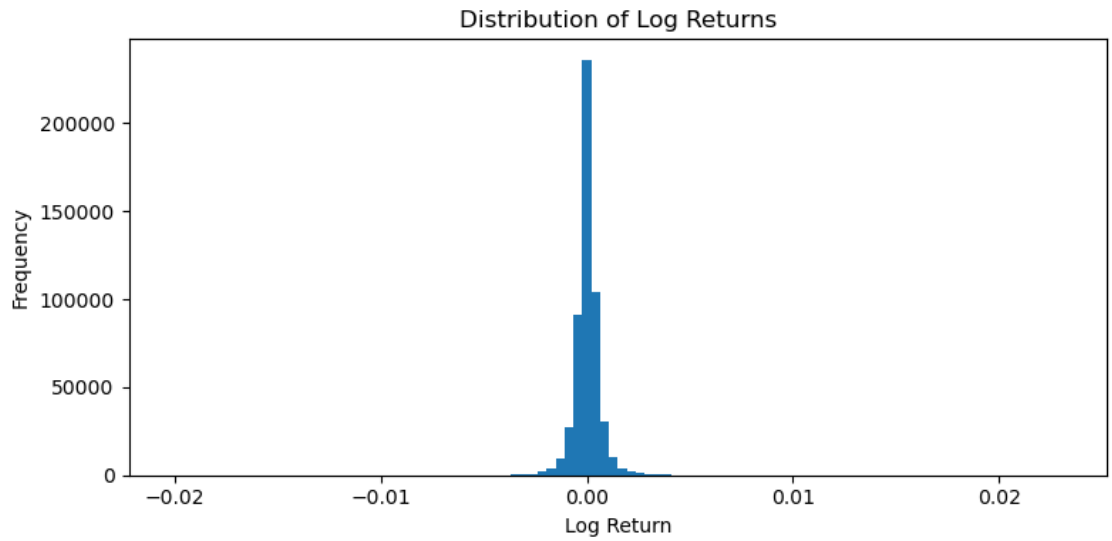
	datetime	close	log_return
0	2025-02-24 00:01	96283.67	0.000026
1	2025-02-24 00:02	96214.67	-0.000717
2	2025-02-24 00:03	96212.25	-0.000025
3	2025-02-24 00:04	96265.17	0.000550
4	2025-02-24 00:05	96321.98	0.000590

4 Modeling Volatility: Data Preparation and Summary

Volatility is not constant over time. Instead, it appears in clusters, where periods of high volatility are followed by more high volatility, and periods of low volatility as well. This motivates the use of models that can adapt to changing uncertainty rather than assuming a fixed level of variance.

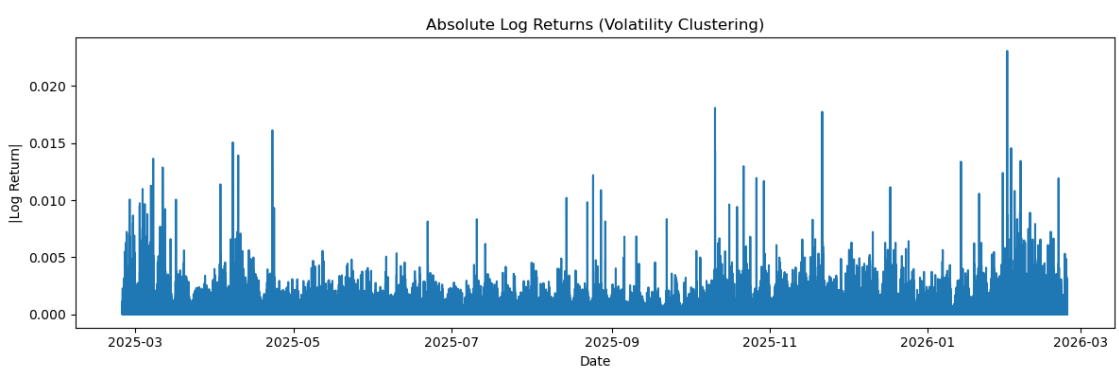
4.0.1 Figure 1 - *Time Series of Log Returns*



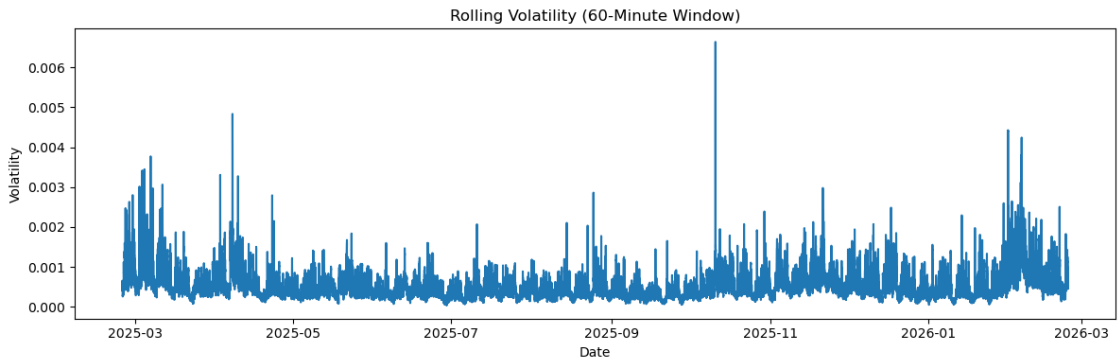


The distribution of returns shows a high concentration of values near zero, along with a number of large positive and negative movements. This pattern is consistent with heavy-tailed behavior, where extreme events occur more frequently than expected under a normal distribution.

4.0.2 Figure 2 - *Rolling Volatility Over Time*



The magnitude of returns shows clear clustering behavior. Large movements tend to occur in groups, followed by periods of relatively small fluctuations. This pattern indicates that volatility is not constant over time, reinforcing the presence of volatility clustering.



Financial markets do not behave consistently over time. Periods of calm can quickly shift into periods of high volatility, making uncertainty constantly change.

Models that assume a fixed level of variability (constant variance) might fail to capture how markets behave. This motivates the use of probabilistic models that can adapt to changing levels of uncertainty over time.

5 Forecasting Setup: Probabilistic Modeling of Returns

5.0.1 Table 3 - Return Summary Statistics

[9]:

	Statistic	Value
0	Count	525217.000000
1	Mean	-0.000001
2	Std Dev	0.000636
3	Min	-0.020089
4	25%	-0.000250
5	Median	0.000000
6	75%	0.000251
7	Max	0.023069

The mean is essentially zero while the variability is small but still present. The data shows large swings, which indicates that volatile change may occur more often than the model would predict

This is why we move toward models that can adapt to both stable and more volatile periods.

We considered two probabilistic forecasting approaches to model the distribution of future returns. A Static Gaussian Model and a Bayesian predictive model.

The intention is for both models to estimate the full distribution of future returns rather than to produce a single point prediction, which allows us to quantify uncertainty and capture the range of possible outcomes

5.0.2 Static Gaussian Model

As a baseline, we assume that returns follow a normal distribution:

$$r_t \sim \mathcal{N}(\mu, \sigma^2)$$

where μ is the mean return and σ^2 is the variance. These parameters are estimated from historical data and remain fixed over time.

This is our baseline model due to it being simple and easy to work with. It does assume that volatility is constant, so it can't respond when market condition change.

5.0.3 Bayesian Predictive Model

To address the constant volatility, we use a Bayesian approach that allows model parameters to update over time. Instead of treating volatility as a constant, we treat it as uncertain and update our beliefs as new data becomes available.

Under this model, parameter uncertainty is incorporated directly into the predictive distribution. As a result, the predictive distribution for the next return takes the form of a Student-t distribution:

$$r_{t+1} \mid \text{data} \sim t_\nu(m_n, s_n)$$

This formula reflects both observed data and parameter uncertainty, allowing the model to adapt to changing volatility and better capture the variability observed in financial markets.

6 Evaluation Design: Model Specification and Parameter Estimation

We use the training data to estimate the average and variability, keeping them fixed during forecasting. This gives us a baseline to compare against the adaptive Bayesian Model.

6.0.1 Table 4 - Static Model Parameter Estimates

[10]:

	Parameter	Value
0	Mean ( )	-0.000001
1	Std Dev ( )	0.000636

7 Results: Forecasting Pipeline

Monte Carlo simulation is used in the forecasting step to generate a range of possible next-step returns rather than a single value. Those simulated returns are then converted into a price samples. Median forecast and the 90% prediction interval summarize the final one-step-ahead output.

7.0.1 Monte Carlo Forecast (Single-Step Example)

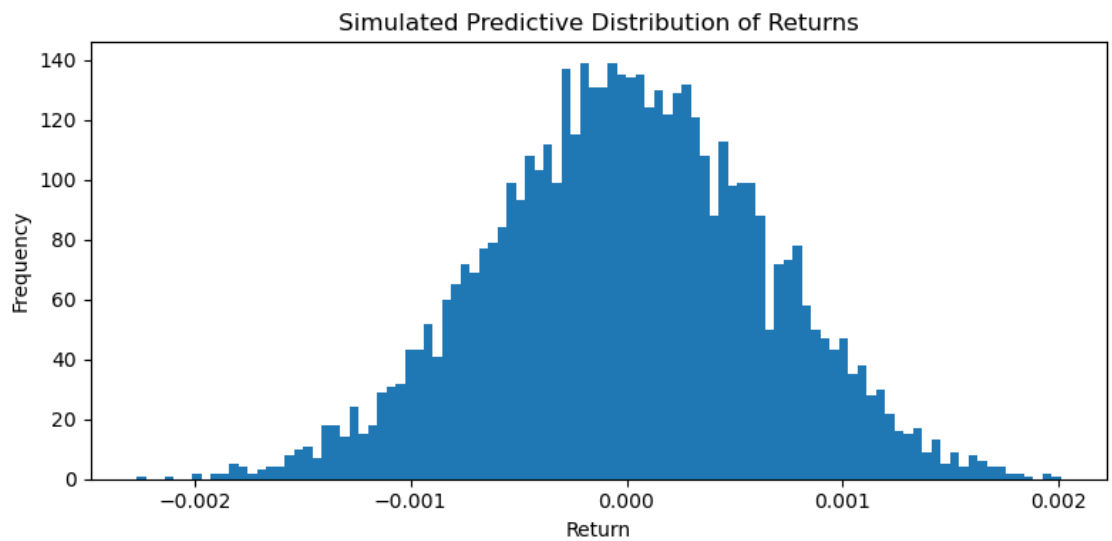
The following example shows how a one-step-ahead predictive distribution is generated. Instead of producing a single forecast, the model simulates many possible outcomes from the predictive distribution.

[11]:

	Metric	Value
0	Median Forecast	0.000004
1	Lower Bound (90% PI)	-0.001051
2	Upper Bound (90% PI)	0.001048

The median of the simulated returns provides a central estimate, while the lower and upper quantiles define a 90% prediction interval. Together, these summarize both expected behavior and forecast uncertainty.

7.0.2 Figure 3 - *Simulated Predictive Distribution of Returns*



The simulated distribution shows the range of possible one-step-ahead returns under the model. Most values are concentrated near the center, while the spread reflects uncertainty. The tails represent less likely but still possible outcomes, creating a more complete view of risk than a single point forecast.

8 Stress Regime Analysis: Walk-Forward Evaluation Design

To evaluate each model's performance, we use a time-based train/test split combined with a walk-forward evaluation. A walk-forward evaluation makes the prediction in order, using only past information at each step, and moves forward through time. This approach avoids using future data and mimics how a real forecasting system operates. This section shows how the data is split, how the models are initialized, and how inputs are prepared before scoring begins.

8.0.1 Train/Test Split

8.0.2 Table 5 - Train/Test Split Summary

[13]:

	Metric	Value
0	Total Observations	525,217
1	Training Observations	420,173
2	Test Observations	105,044
3	Training Start	2025-02-24 00:01
4	Training End	2025-12-13 01:00
5	Test Start	2025-12-13 01:01
6	Test End	2026-02-24 00:00

8.0.3 Model Initialization

The static Gaussian model is fit once on the training data and remains fixed throughout the evaluation period.

The Bayesian model is initialized using the training data and updated sequentially as new observations become available during the test period.

8.0.4 Table 6 - Model Initialization

[14]:

	Model	Parameter	Value
0	Static Gaussian	Mean / Std Dev	=-0.000000, =0.000611
1	Bayesian Predictive	/ /	=420,173, =-0.000000, =0.000611

8.0.5 Table 7 - Model Input Structure

[15]:

datetime	S_tm1	S_t	r_t
2025-02-24 00:02	96283.67	96214.67	-0.000717
2025-02-24 00:03	96214.67	96212.25	-0.000025
2025-02-24 00:04	96212.25	96265.17	0.000550
2025-02-24 00:05	96265.17	96321.98	0.000590
2025-02-24 00:06	96321.98	96341.23	0.000200

9 Conclusion: Results

We compare both models with unseen data, using walk-forward evaluation. We focus on how well each model captures uncertainty using metrics like calibration, interval width, log score, CPRS, and median absolute error. Our main question is simple. Does the adaptive model give a better, more realistic view of uncertainty, especially when the market shifts?

9.0.1 Table 8 - Walk-Forward Performance

[23]:

	Metric	Static Gaussian	Bayesian Predictive
0	Coverage (90%)	89.65%	89.81%
1	Average Interval Width	167.2950	167.4064
2	Median Absolute Error (MAE)	21.3052	21.2933
3	Average Log Score	5.7746	5.7827
4	Average CRPS	27.9924	27.9633

The main metrics we use are:

- **Mean CRPS**: overall probabilistic accuracy, lower is better
- **Coverage (90%)**: how often the true price falls inside the 90% prediction interval
- **Median Absolute Error**: the typical error of the median forecast
- **Average Log Score**: how much probability the model assigns to what happened

9.1 Continuous Ranked Probability Score (CRPS)

CRPS measures how good the entire predicted distribution is, not just a single point estimate. It rewards models that are both accurate and assign probability to the right areas.

In practice, we approximate it using Monte Carlo samples from the predictive distribution.

[24]:

	Model	Mean CRPS	Coverage (90%)	Median Abs Error
0	Static Gaussian	27.99	0.896	21.31
1	Bayesian Predictive	27.96	0.898	21.29

Across the full test set, the two models perform almost identically. The Bayesian model shows slight improvements in probabilistic metrics like CRPS and coverage, but the gains are marginal, and point accuracy remains essentially unchanged.

These differences are small. The Bayesian approach provides slightly better estimates of uncertainty. However, it does not meaningfully improve overall predictive performance. Based on the performance, in this setting, adaptive volatility modeling offers limited practical benefits.

The complexity leads to marginal improvements in uncertainty but does not change the broader forecasting outcome. In a high-noise cryptocurrency environment, much of the next-step variation remains difficult to predict, which limits how much improvement any model can achieve at these intervals.

9.2 Limitations

- The assumption of the model is that over short time windows, the returns are normally distributed. If behavior changes extremely quickly or exhibits more heavy-tailed behavior, this assumption can break down.
- Rather than modeling things like volatility clustering or regime shifts, the volatility is only updated using Bayesian rules.

- The Bayesian approach may not update fast enough during sudden volatility spikes, which can underestimate uncertainty temporarily.

Given these known limitations, it captures the underlying behavior but still simplifies several aspects of how actual financial markets operate.

9.3 Stress Period Analysis

Full-sample averages can mask where the model actually fails, especially during more volatile periods.

To fully review our models, we break out performance during high-volatile periods and evaluate each model separately.

The intention behind this diagnostic is that a model can look acceptable on average but can be overconfident when market conditions are unstable.

This answer is a more direct research question. Not if the Bayesian update improves overall, but if it can predict correctly when the market becomes volatile.

9.3.1 Table 9 - Performance by Market Regime

[26]:
```
   subset   model       n  coverage_90  avg_interval_width  mean_CRPS  \
0    calm   bayes   94539        0.934              169.11      23.50
1    calm  static   94539        0.933              169.13      23.52
2    full   bayes  105044        0.898              167.41      27.96
3    full  static  105044        0.896              167.30      27.99
4  stress   bayes   10505        0.572              152.04      68.14
5  stress  static   10505        0.567              150.74      68.22

   mean_log_score  median_abs_error
0            6.10             19.19
1            6.09             19.16
2            5.78             21.29
3            5.77             21.31
4            2.97             63.28
5            2.91             63.37
```

[27]:
```
#TODO: START HERE
```

During calm periods, both models are identical. Coverage is around 93% for both; CRPS is low, and the median absolute error is nearly identical. In stable conditions, Bayesian updating adds complexity but only delivers a small benefit.

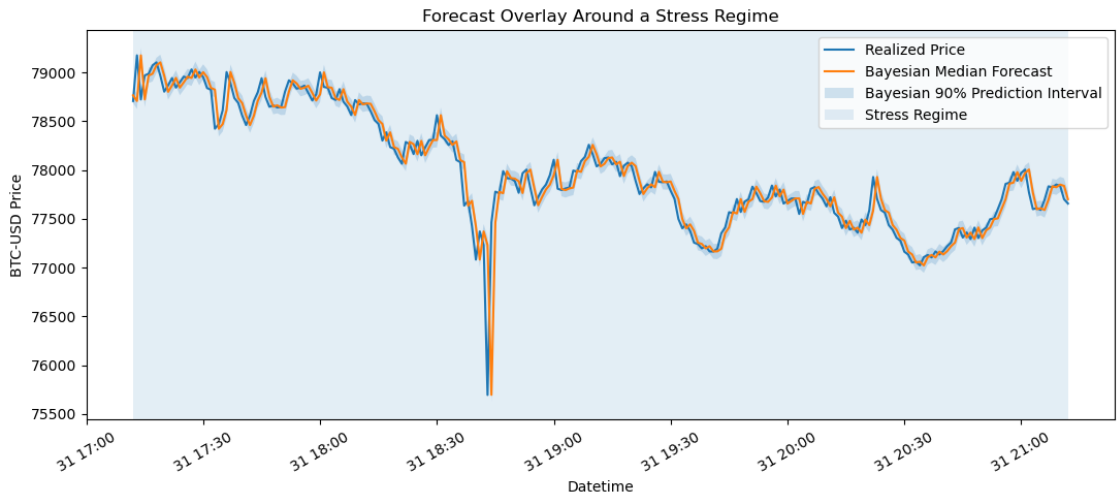
Across the full sample, the same pattern holds. The Bayesian model is only slightly better on probabilistic metrics, and the improvement is negligible.

Averages across the full sample don't show the full story, especially when the model is under stress. Coverage drops from around 90% to roughly 57% for both models, while CRPS and median absolute error increase. This means the 90% prediction intervals often miss the outcome.

The interval widths do not widen enough to reflect the increase in uncertainty, so the models stay overconfident when that matters the most. During volatility spikes, both models are significantly underestimating risk

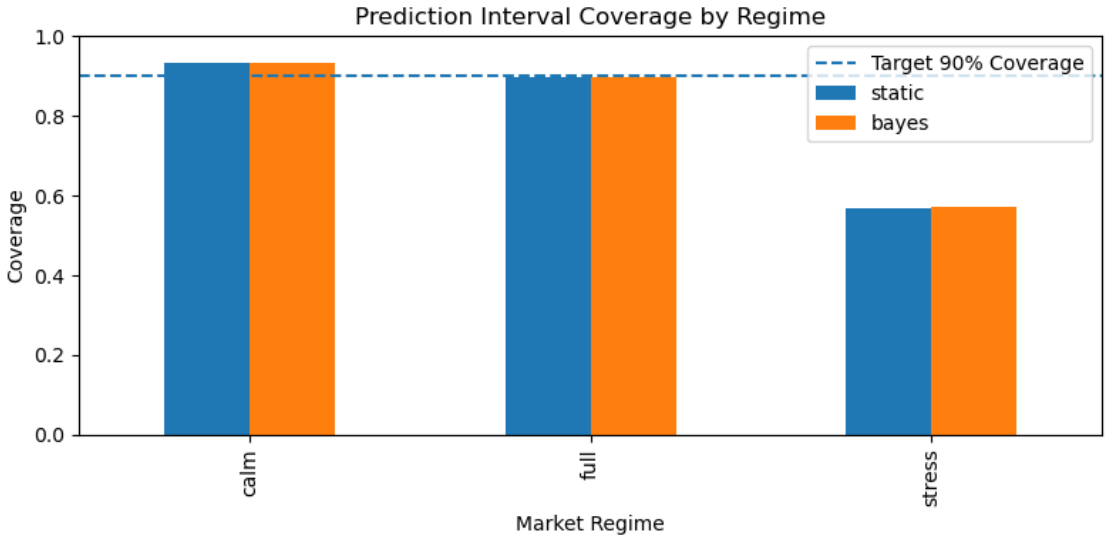
The Bayesian model performs slightly better than the static model. However, both models break down during volatility shifts, and the Bayesian updating process does not adjust fast enough to prevent it.

9.3.2 Figure 4 - *Forecast Overlay During a Stress Window*



The stress-window overlay visually shows failure. The realized prices move quickly, but the predictive model doesn't widen fast enough to remain accurate. That gap is the coverage collapse shown in the table above. Looking at an average to determine model fit isn't enough, and stress results matter. It shows that the model is overconfident during periods of highest uncertainty.

9.3.3 Figure 5 - *Prediction Interval Coverage by Market Regime*



10 Discussion and Conclusion

In this comparison, neither model clearly outperforms the other. Adaptive volatility learning provides only marginal improvements, while both models remain overconfident during high-volatility periods.

The static model is simple and competitive. The Bayesian model is more complex and does show slightly better probabilistic metrics, but the gains are not large enough to address the core issue revealed by the stress analysis.

At its core, the problem is reactivity. The Bayesian model updates using past returns, so it adapts over time, but it can only react after volatility has already entered the data.

During abrupt regime shifts, this becomes a real limitation. One change is not enough to fully adjust the model, so it takes multiple steps to catch up.

What matters most in environments like this is not small improvements in average CRPS or similar metrics. The system needs to remain robust and correctly estimate uncertainty when the market becomes unstable.

This project does not show that Bayesian updating meaningfully outperforms a static baseline. It shows something more useful: adaptive updating helps slightly, but both models underestimate the tails of the distribution and break down under stress.

Understanding where models fail, and why, is critical in real-world settings. These results point toward future work on faster-reacting volatility models, as well as approaches that better capture volatility clustering and regime changes.